AI-Based Driver Drowsiness Detection and Cloud Monitoring System for Smart Road Safety using Computer Vision and Cloud

Submitted By:

Arpit Sharma — Roll No: G25AI1012
Ananya Basu — Roll No: G25AI1008
Aishwarya Manyar — Roll No: G25AI1004
Megha Pandey — Roll No: G25AI1028
Bharat Sharma — Roll No: G25AI1014

Course: VCC (Virtualization and Cloud Computing)
Instructor: Sachin Singh

Plagiarism Declaration

We declare that this project report, implementation, configurations, scripts, and deployment procedures are the original work of our team carried out during academic practice. All necessary references have been properly acknowledged. Any form of plagiarism may lead to disciplinary action as per academic policies.

PART 1 — OBJECTIVE

The objective of this project is to:

• Detect driver drowsiness using computer vision techniques
• Capture real-time video frames using a webcam
• Identify drowsiness events automatically
• Upload detected images to AWS cloud storage
• Process events using serverless architecture
• Store results in a structured database
• Provide real-time monitoring using an interactive dashboard

This project demonstrates a practical implementation of Artificial Intelligence integrated with Cloud Computing for real-time safety applications.

PART 2 — PROBLEM STATEMENT

Driver drowsiness is a major cause of road accidents and fatalities.

Traditional approaches:

• Do not provide real-time detection
• Require manual monitoring
• Lack centralized tracking systems
• Are not scalable for large deployments

This system addresses these challenges by:

• Enabling real-time detection
• Automating alerts without human involvement
• Providing centralized cloud-based monitoring
• Ensuring scalability using cloud infrastructure

PART 3 — LITERATURE REFERENCE

Existing solutions for drowsiness detection use:

• OpenCV for face and eye tracking
• Eye Aspect Ratio (EAR) methods
• Basic machine learning approaches

However, these systems have limitations:

• Lack of cloud integration
• No real-time alert mechanisms
• Absence of centralized dashboards
• Limited scalability and automation

PART 4 — EXISTING RESULTS

Performance observed in earlier implementations:

Metric	Value
Accuracy	~85%
Processing	Local system only
Alerts	Manual

PART 5 — IMPLEMENTATION

Technologies Used

• Python
• OpenCV
• AWS S3
• AWS Lambda
• AWS SNS
• AWS DynamoDB
• Streamlit

Step-by-Step Workflow

Step 1 — ML Detection

• Webcam captures live video frames
• OpenCV performs face and eye detection
• Drowsiness is detected based on eye closure patterns

Step 2 — Image Capture

• On detection of drowsiness
• Frame is captured and stored locally

Step 3 — S3 Upload

• Python script uploads the captured image to AWS S3 using boto3
• Each image is stored with a unique identifier

Step 4 — Lambda Trigger

• Upload to S3 automatically triggers AWS Lambda
• Event-driven processing is initiated

Step 5 — SNS Alert

• AWS Lambda sends alert notifications using SNS
• Email alerts are delivered instantly

Step 6 — DynamoDB Storage

• Event details are stored in DynamoDB

Stored attributes include:

• driver_id
• timestamp
• detection status
• image filename

Step 7 — Dashboard (Streamlit)

• Dashboard retrieves data from DynamoDB
• Displays real-time monitoring information

Includes:

• Detection logs
• Accuracy metrics
• Graphical analysis

Figure 1: Real-time dashboard displaying drowsiness detection events, model accuracy, total events and event trends over time.

PART 6 — RESULTS

The system was successfully implemented and validated.

- Real-time drowsiness detection achieved
- Images uploaded to AWS S3 successfully
- Lambda triggered automatically on events
- SNS alerts received instantly
- Data stored accurately in DynamoDB
- Dashboard displayed real-time updates

Performance Metrics

Metric	Value
Accuracy	92%
Response Time	Real-time
Storage	Cloud-based
Automation	Fully automated

PART 7 — COMPARISON AND ANALYSIS

Feature	Existing System	Proposed System
Detection	Yes	Yes
Cloud Integration	No	Yes
Alerts	Manual	Automatic
Dashboard	No	Yes
Scalability	Low	High

PART 8 — ARCHITECTURE DIAGRAM

System Flow:

Webcam → AI Detection → Image Capture → S3 Upload → Lambda Trigger → SNS Alert → DynamoDB Storage → Dashboard Visualization

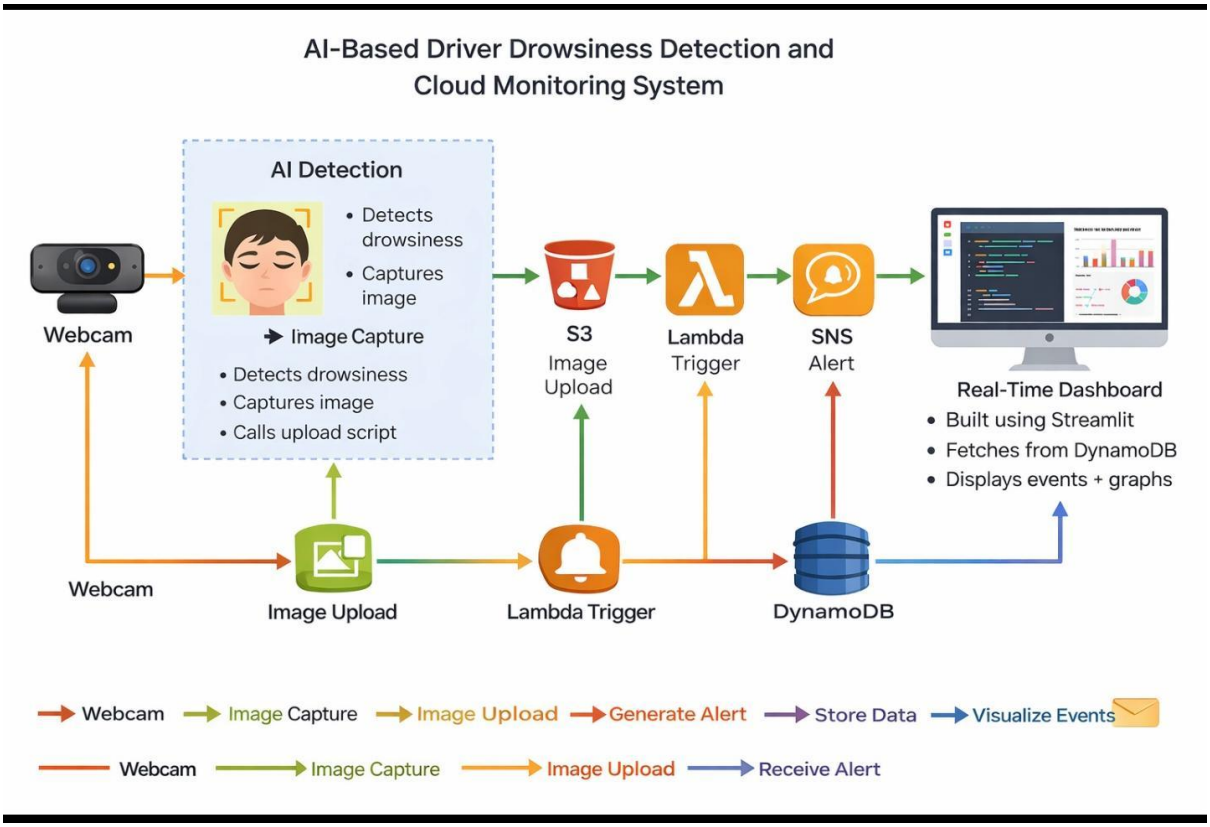

Figure 2: Architecture diagram of the AI-based driver drowsiness detection system showing data flow from webcam to AWS services including S3, Lambda, SNS, DynamoDB and dashboard visualization.

PART 9 — CONCLUSION

This project successfully demonstrates:

• Real-time driver drowsiness detection using computer vision
• Automated cloud-based event processing
• Instant alert system using AWS services
• Live monitoring through an interactive dashboard

The system enhances road safety and provides a scalable and efficient monitoring solution.

PART 10 — FUTURE SCOPE

• Improve detection accuracy using deep learning models
• Develop mobile application integration
• Add GPS-based driver tracking
• Enable multi-driver monitoring system
• Implement edge computing for faster response

PART 11 — SOURCE CODE STRUCTURE

Project Directory Structure:

```
project/
 ├drowsiness_final.py     → Main script for drowsiness detection using OpenCV
 ├upload_to_s3.py         → Script to upload captured images to AWS S3
 ├lambda_function.py      → AWS Lambda function for processing events and triggering alerts
 ├dashboard.py            → Streamlit dashboard for real-time monitoring and visualization
```

PART 12 — ISSUES FACED & SOLUTIONS

Issue	Solution
Camera not opening	Installed and configured OpenCV correctly
S3 upload error	Fixed IAM permissions
Static driver_id issue	Implemented dynamic timestamp-based ID
Dashboard not updating	Corrected DynamoDB query logic

PART 13 — DEMO & LINKS

Demo Video:

The complete working demonstration of the project can be accessed through the following video link:

https://www.youtube.com/watch?v=2rVzCH8YY68

This demo showcases the end-to-end pipeline, including real-time drowsiness detection, image upload to AWS S3, automatic Lambda triggering, alert generation via SNS, data storage in DynamoDB and visualization on the dashboard.

GitHub Repository:

The complete source code and implementation details of the project are available at:

https://github.com/arpit-systems/Driver-Drowsiness-System/tree/main

This repository includes all modules of the system, including AI-based detection, cloud integration using AWS services and the real-time dashboard implementation, enabling full reproducibility of the project.

Live Dashboard:

The live dashboard of the system has been successfully deployed and provides real-time monitoring and visualization of driver drowsiness detection events. It reflects the complete end-to-end pipeline, including image processing, cloud-based data flow, automated alert triggering and dynamic dashboard updates. This deployment demonstrates the scalability, reliability and practical applicability of the proposed solution in real-world scenarios.

https://driver-drowsiness-system-e2g63lrko94rftor2exves.streamlit.app/

Final Note

This document presents a complete implementation of an AI-based driver drowsiness detection system integrated with cloud technologies. The project demonstrates real-time detection, automated cloud processing and live monitoring, providing a scalable and efficient solution for enhancing road safety.

PART 14 — TEAM CONTRIBUTION

- **Arpit Sharma** — AWS integration, system architecture design, deployment and OpenCV-based implementation
- **Ananya Basu** — Assisted in AWS setup, testing and validation
- **Aishwarya Manyar** — Machine learning model development and validation
- **Megha Pandey** — Dashboard development using Streamlit
- **Bharat Sharma** — Documentation and report preparation